Москва

07.10.2026



Разбор технологий HMI, xiObj и TPZ в SAP PI

Илья Кузнецов
Консультант по интеграции, Август

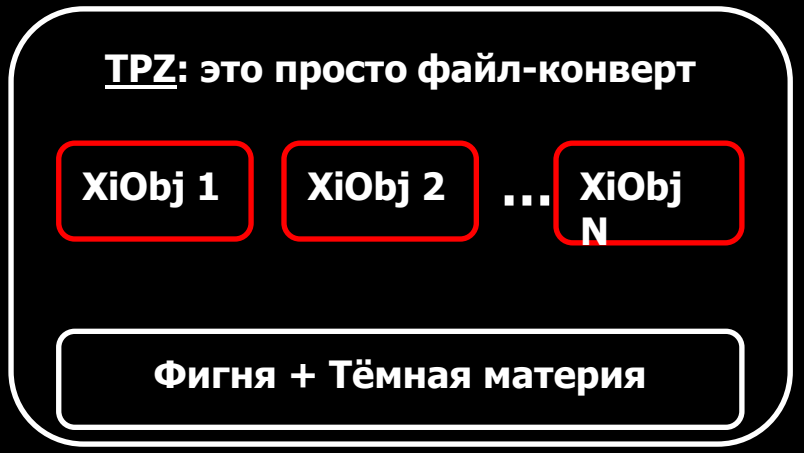
avgust
crop protection

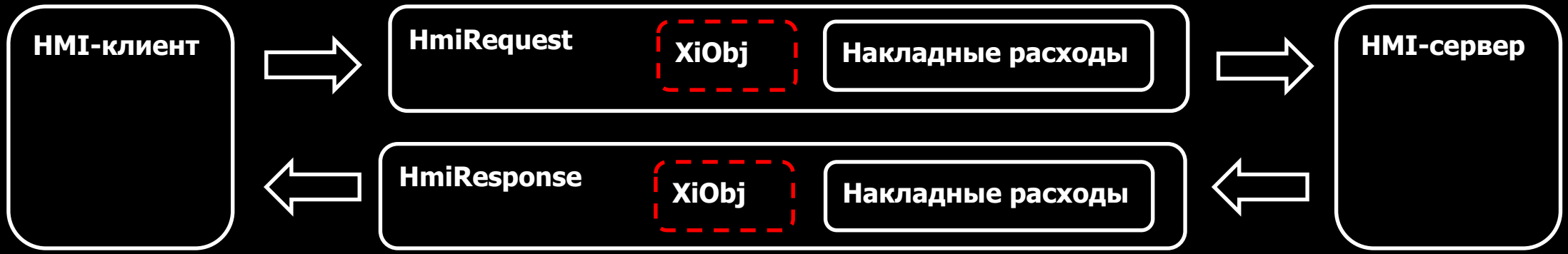
Предисловие. Подход к снаряду.



- Презентация в основном для SAP PIщиков (краешком CPI)
- Лучше если Вы – ява-разработчик, любого уровня. Так сможете использовать готовые наработки хотя бы в IDE
- Хотя глина старая, гончарный круг современный: JDK25, JAXB4 и без библиотек от вендора
- Фокус пока на бэкенде, нет фронтенда

О чём вообще речь в докладе?





TPZ, реверс-инжиниринг



TPZ, устройство

ZIP-конверт содержит:

1. Metadata.properties – Java properties, для всех
2. metadata.esr.710 – Java properties, только ESR
3. TPT [0..1] – полная выгрузка XiObj’ектов
4. DLT [0..n] – дельта-выгрузка, только ESR; для 7.1 и 7.3 могут быть разные варианты
5. META-INF – только из ESR, дескрипторы развёртывания и манифесты

Парсер: TpzContainer.java

TPT / DLT – начинается интересное



1. Секретный GUID DCF68E99-B34D-47A1-8F8F-B18F2AE8E250 инициализирует контрольную сумму
2. Контрольные суммы могут быть и внутри XML-каталога (на рисунке это не стал показывать)
3. Длина BLOB – 0x10000 (1МБ)
4. Все BLOBы после проверки CRC32 надо склеить воедино и только затем оттуда вытаскивать вкусное
5. XML-каталог хранит смещения фрагментов внутри целевого BLOB
6. TPT и DLT устроены одинаково

Парсер: TptFragments.java

TPT-сегменты распакованные из BLOB

XML-каталог ссылается на сегменты примерно так:

Типы сегментов

1. “header”, “modelGeneralData”, “metaModel-Model” это служебные динамические xml
2. metaModel-PVC это статический XML
3. “modelGeneralData” это искомые XiObj
4. Прочие это тёмная материя (грустно и невкусно)

Найденная удобная последовательность разбора ява-кодом



1. Распаковываем TPZ как ZIP во временную папку %TEMP%/Zatupka_12
2. Определяем, какой TPT или DLT будем брать, глядя на Metadata.properties или имена файлов
3. Делаем общий BLOB-файл из TPT/DLT, также catalog.xml, проверяем CRC
4. Делаем отдельные XML/BIN файлы из BLOB
5. Можно все получившиеся XML-файлы проверить на валидность, не должно быть двоичного мусора. Начинку XiObj сразу лучше не парсить.

Протестировано на всех доступных мне TPZ-файлах, включая максимальный XI7_3_S4CORE_106_02_00.tpz 238МБ. И на огромных текстах с пробелами на десятки МБ чтобы увидеть мусор среди чистого.

TPZ, итоги



Выглядит так, что чтение и разбор содержимого на верхнем уровне работает.

Пример стресс-тестов:
Сперва разбираем кучу TPZ на куски, а затем парсим отдельные XiObj (забегая вперёд)
Проверка подсчёта CRC из папки TPT-файлов

Что мне помогло в разборках? NWDS, дистрибутив SAP PO и Far Manager, а также IdeaJ в многочисленных тестах. Пробовал также Kaitai и ANTLR, но ява-тесты оказались проще.

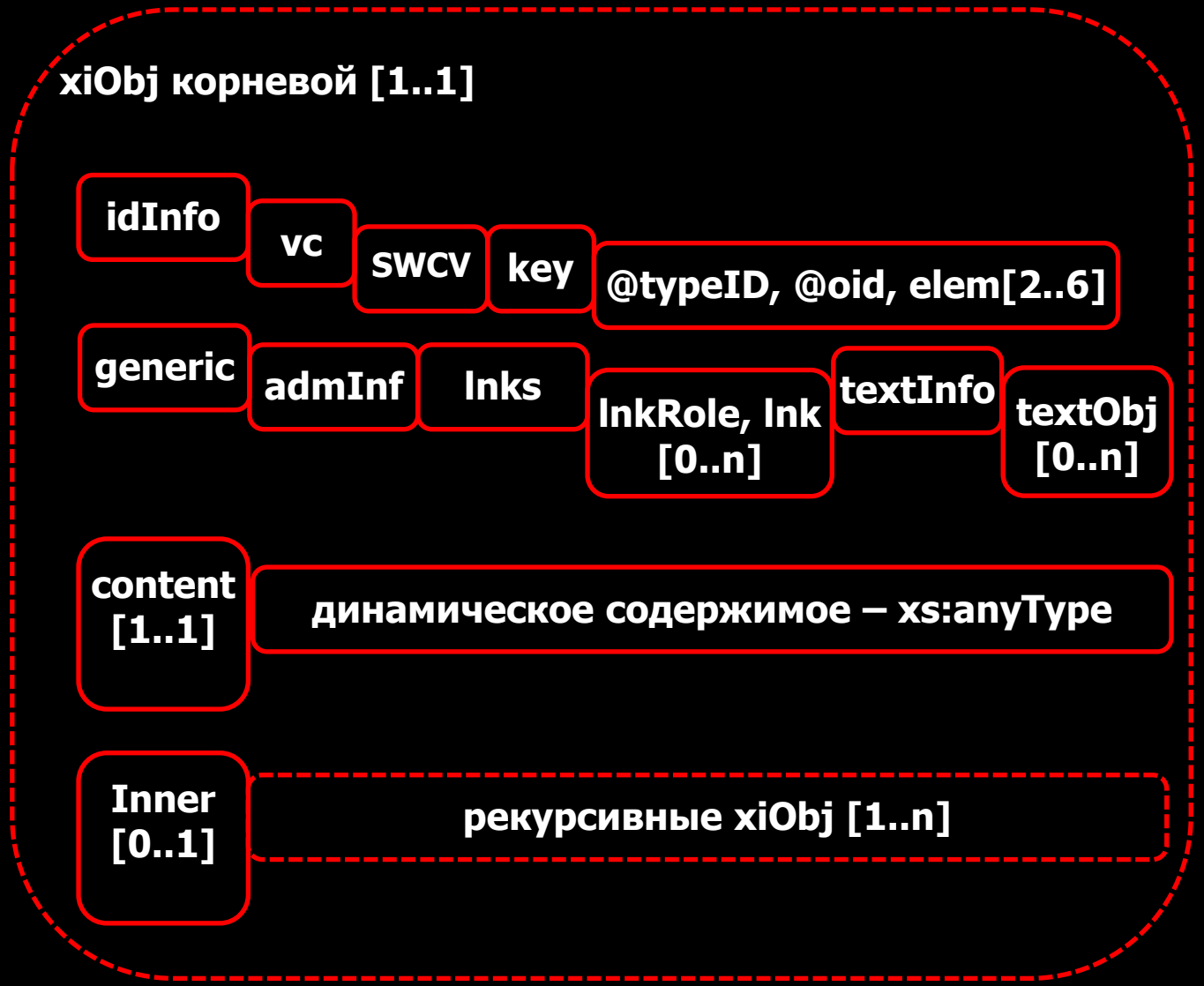
Вопросы по TPZ?

XiObj – наш немецкий слоняра примерно 2005гр. Жив поныне.



```
<p1:xiObj xmlns:p1="urn:sap-com:xi">
  <p1:idInfo VID="5279eaab342211e9a6df00000036471e">
    <p1:vc vcType="D">
      <p1:clCxt consider="A"/>
    </p1:vc>
    <p1:key typeID="ValueMapping" oid="3030af30f92b3286af5c230de18a2d8b">
      <p1:elem>a43390b0-3421-11e9-9ed0-c85b76a80a70</p1:elem>
    </p1:key>
  </p1:idInfo>
  <p1:generic>
    <p1:admInf>
      <p1:modifBy>701102</p1:modifBy>
      <p1:modifAt>2019-02-19 13:10:44.618</p1:modifAt>
      <modifAtLong>1550567444618</modifAtLong>
      <p1:owner>701102</p1:owner>
    </p1:admInf>
    <p1:lnks>
      <p1:lnkRole role="PARENT_FOLDER" kpos="0">
        <p1:lnk rMode="R">
          <p1:vc vcType="D">
            <p1:clCxt consider="A"/>
          </p1:vc>
          <p1:key typeID="FOLDER" oid="1af3784e2b7811e988f700000036471e">
            <p1:elem>1ab816282b7811e99141000000036471e</p1:elem>
          </p1:key>
        </p1:lnk>
      </p1:lnkRole>
    </p1:lnks>
    <p1:textInfo>
      <p1:textObj id="5279eaab342211e9a6df00000036471e" type="0" masterL="EN"/>
    </p1:textInfo>
  </p1:generic>
  <p1:content>
    <Valuemapping groupid="a43390b0-3421-11e9-9ed0-c85b76a80a70" groupName="POF075">
      <Representation>
        <system>ERP</system>
        <objectTypeName>ZZPORT</objectTypeName>
        <value>LATTAKIA</value>
      </Representation>
      <Representation>
        <system>C4C</system>
        <objectTypeName>PortOfDischarge</objectTypeName>
        <value>075</value>
      </Representation>
    </Valuemapping>
  </p1:content>
</p1:xiObj>
```

XiObj – общая структура



Вероятно, появился в XI 3.0, не документирован официально, используется даже в SAP CPI (с мелкими изменениями в областях имён).

В отличии от двоичных сегментов TPZ, может быть создан любым инструментом.

XSD созданный вручную.

XiObj, конвейер парсинга



1. Получаем xiObj из TPZ либо иным способом (SimpleQuery, HMI, из CPI, …)
2. Парсим через JAXB статическое содержимое (включая рекурсивные inner xiObj)
3. Динамический контент парсим, получая org.w3c.dom.Element[]
4. Из org.w3c.dom.Element[], глядя на @typeID, получаем ещё более вкусный прикладной POJO

Парсер: XiObjParser.java

XiObj, что уже вытаскивается динамического



```
111             try {
112                 Object payload = switch (typeID) {
113                     case "AllInOne" -> parseAllInOne(src);
114                     case "Channel" -> parseChannel(src);
115                     case "Party" -> parseParty(src);
116                     case "Service" -> parseService(src);
117                     case "AlertRule" -> parseAlertRule(src);
118                     case "FOLDER" -> parseFolder(src);
119                     case "ValueMapping" -> parseValueMapping(src);
120                     case "OutboundBinding" -> parseOutboundBinding(src);
121                     case "RoutingRule" -> parseRoutingRule(src);
122                     case "InboundBinding" -> parseInboundBinding(src);
123                     case "swc" -> parseSwc(src);
124                     case "namespdecl" -> parseNamespaces(src);
125
126                     case "ifmtypedef" -> parseType(src);
127                     case "ifmextdef" -> parseExtdef(src);
128                     case "ifmmessage" -> parseMessageType(src);
129                     case "ifmmessif" -> parseServiceInterface(src);
130                     case "ifmfaultm" -> parseFault(src);
131                     case "ifmcontobj" -> parseContextObject(src);
132                     case "RepBProcess" -> parseDefinition(src);
133                     case "process" -> parseProcess(src);
134
135                     case "ChannelTemplate" -> parseChannelTemplate(src);
136                     case "AdapterMetaData" -> parseAdapterMetaData(src);
137                     case "XI_TRAFO" -> parseXiTrafo(src);
138                     case "MAPPING" -> parseMapping(src);
139                     case "TRAFO_JAR" -> parseArchive(src);
140                     case "IFLOW", "MappingRelation", "MAP_TEMPLATE", "RoutingRelation", "ifmtypeenh", "FUNC_LIB",
141                         "AlertCategory" -> Object.class;
142                     case "idoc", "rfc", "type", "imsg", "iseg", "ityp" -> parseXmlcontent(src);
143                     case "ifmuitexts", "ariscommonfile", "arisreport", "arisfilter", "aristemplate", "arisfssheet",
144                         "arismodel", "arisobjdef", "ariscxndef", "aristextdef" -> new Object();   //TODO
145                     default -> typeID;
146                 };
```

Directory:

AllInOne, Channel, Party, Service, AlertRule, ValueMapping, OutboundBinding, RoutingRule, InboundBinding;

Конфигурационные сценарии имеют лишь ссылки.

Общее:

FOLDER, Тексты;

Repository:

Namespaces, DT, MT, SI, ED, Faults, ContextObjects. Меппинги, Imported Archives, метаданные адаптеров и шаблоны каналов.
И зачем-то процессы, для красоты ☺

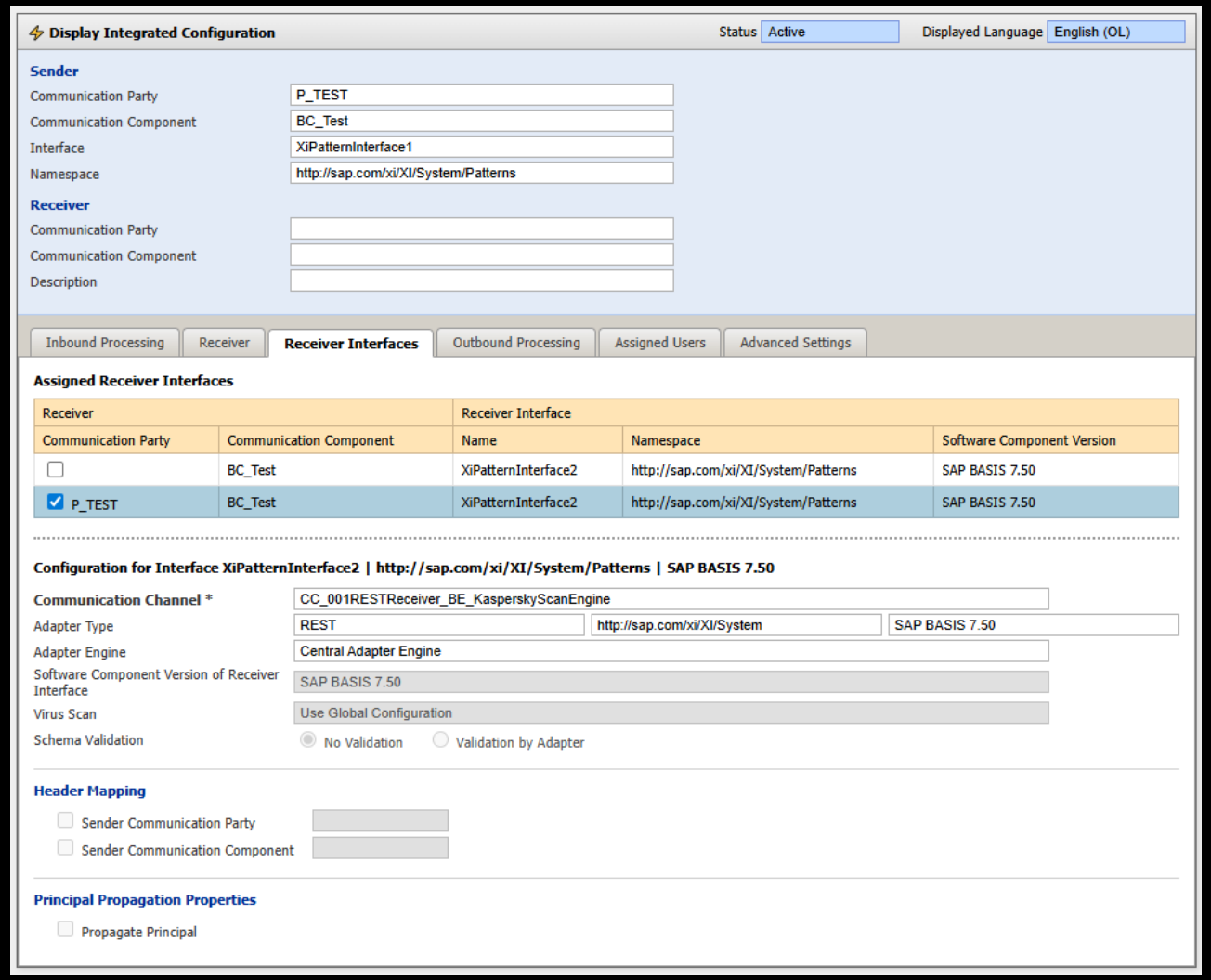
Всё остальное -- //TODO, есть нюансы в разборе айдоков (старый формат с отдельными сегментами и новый в едином WSDL)

XiObj, пока не итоги, всё ещё развивается



- Весь доступный XI CONTENT парсится, это сотни мегабайт
- Под руками нет CPIных примеров, в них есть нюансы неймспейсов (помню по ранним попыткам их анализа)
- Распарсить на уровне JAXB легко. Дальше сразу требуется разбор XSD и WSDL (для Repository) или много буков для ICo AllInOne, где нюансы RD/ID и сплитов и HeaderMapping’ов
- Очень полезным инструментом оказался TRANG, позволяет из сотен XML сделать XSD в батнике
- Современный JAXB 4 стал мощнее, и не сдаёт позиции
- Eclipse XSOM позволяет работать с XSD схемами но ссылки и импорты в ifr:// протоколе немецкого вендора пишем сами

XiObj, что хотелось бы сделать



На входе пачка TPZ или коннект к SAP PO, на выходе html-документация, полученная из ESR + DIR.

На входе пачка TPZ, суём их в поисковую мясорубку, имеем сервис поиска по стандартному контенту от вендора или с предыдущих проектов (аналогично с коннектом к SAP PO).

На входе пачка TPZ или коннект к разным SAP PO, на выходе KartaDSL для запуска цифрового двойника.

Генерация документации

На входе пачка TPZ или коннект к SAP PO, на выходе html-документация, полученная из ESR + DIR.

Сперва разбираем кучу TPZ на куски, получаем много мелких XML-файлов в %TEMP%/Zatupka_1234567/*.xml
Затем парсим их как xiObj верхнего уровня, парсим динамическое содержимое.
Дальше через ANTLR ST или иной шаблонизатор генерируем HTML.

Поисковая мясорубка

На входе пачка TPZ или коннект к SAP PO.

Сперва разбираем кучу TPZ на куски, получаем много мелких XML-файлов в %TEMP%/Zatupka_1234567/*.xml, либо через HMI / SimpleQuery читаем xiObj по http.

Затем суём их в Эластик или даже встроенный Apache Lucene, обновляем поисковый индекс, «и всё заверте...» (с) Аверченко

Вопросы по XiObj?

HMI, синдром неприятия чужой разработки
(*Not Invented Here* NIH)



Вероятно, появился в XI 3.0, не документирован официально

POST

http://sappo:50000/rep/getregisteredhmimethods/int?container=any

	A	B	C	D	E	F
1	serviceid	methodid	release	subrelease	SP	patchlevel
2	_subscription	getdeliverychannels	7.3	*	*	*
3	_subscription	getdcvariants	7.3	*	*	*
4	_subscription	_unsubscribe	7.3	*	*	*
5	_subscription	getdcvmetadata	7.3	*	*	*
6	_subscription	_subscribe	7.3	*	*	*
7	_subscription	getsubscriptionsforobject	7.3	*	*	*
8	_subscription	getemailforuser	7.3	*	*	*
9	_subscription	editsubscription	7.3	*	*	*
10	_subscription	getdcmetadata	7.3	*	*	*
11	_subscription	getsubscriptionsforuser	7.3	*	*	*
12	acl	default	7.3	*	*	*
13	applcomp	content_languages	*	*	*	*
14	applcomp	support_package	*	*	*	*
15	applcomp	release	*	*	*	*
16	applcomp	level	*	*	*	*
17	auth	haspermissions	7.0	*	0	*
18	cads	default	7.3	*	*	*
19	changelistpartialservice	retrieveobjectcontainer	7.0	*	0	*
20	changelistpartialservice	_moveacrosschangelist	7.0	*	0	*
21	changelistpartialservice	_changelistpartialmethod	7.0	*	0	*
22	checkmmeswcv	default	7.3	*	*	*
23	checkobjectsserviceid	picchecks	*	*	*	*
24	checkobjectsserviceid	checkactiveobject	*	*	*	*
25	checkobjectsserviceid	checkobjectsmethodid	*	*	*	*
26	checkobjectsserviceid	clusers	*	*	*	*
27	checkobjectsserviceid	changelist	*	*	*	*
28	checkservices	checkobjectsmethodid	*	*	*	*
29	checkservices	checkmatchinginterfacesassignmentmethodid	*	*	*	*
30	classificationservice	_getclassificationsystemsgreaterthanversion	7.0	*	0	*
31	classificationservice	_getclassificationsystemvalues	7.0	*	0	*
32	cmstransport	transport	*	*	*	*
33	compiler_service_id	compiler_method_id	7.0	*	0	*
34	conntest	start_test	*	*	*	*
35	conntest	get_last_info	*	*	*	*
36	conntest	cleanup	*	*	*	*
37	conntest	about_last_test	*	*	*	*
38	containeraccess	getcontainer	*	*	*	*
39	containeraccess	getcontainers	*	*	*	*

Для SAP PO 7.5sp35 получаем 170 методов Repository.

У нас есть HMI-приложение /rep, у него есть сервисы с методами, рассчитывающие на все (звёздочка) или определённые (вида 7.3 и выше) релизы.

Формально, некоторые сервисы требуют сессионности (в структуре предусмотрено).

Быстрый взгляд на instance и HmiRequest



```
▼<instance typeid="com.sap.aii.util.hmi.core.msg.HmiRequest">
 ▼<attribute isleave="true" leave_typeid="string" name="ClientId">
    <value index="0" isnull="false">74f9eda97c2211f1a63500155d001a19</value>
  </attribute>
 ▼<attribute isleave="true" leave_typeid="string" name="ClientLanguage">
    <value index="0" isnull="false">EN</value>
  </attribute>
 ▼<attribute isleave="false" name="ClientLevel">
  ▼<value index="0" isnull="false">
   ▼<instance typeid="com.sap.aii.util.applcomp.ApplCompLevel">
    ▼<attribute isleave="true" leave_typeid="string" name="Release">
       <value index="0" isnull="false">7.3</value>
      </attribute>
     ▼<attribute isleave="true" leave_typeid="string" name="SupportPackage">
       <value index="0" isnull="false">*</value>
      </attribute>
     </instance>
    </value>
  </attribute>
 ▼<attribute isleave="true" leave_typeid="string" name="ClientPassword">
    <value index="0" isnull="false">dummy</value>
  </attribute>
 ▼<attribute isleave="true" leave_typeid="string" name="ClientUser">
    <value index="0" isnull="false">poduser</value>
  </attribute>
 ▼<attribute isleave="true" leave_typeid="string" name="ControlFlag">
    <value index="0" isnull="false">0</value>
  </attribute>
 ▼<attribute isleave="true" leave_typeid="string" name="HmiSpecVersion">
    <value index="0" isnull="false">1.0</value>
  </attribute>
 ▼<attribute isleave="true" leave_typeid="string" name="MethodId">
    <value index="0" isnull="false">default</value>
  </attribute>
 ▼<attribute isleave="false" name="MethodInput">
    <value index="0" isnull="true"/>
  </attribute>
 ▼<attribute isleave="true" leave_typeid="string" name="RequestId">
    <value index="0" isnull="false">74f9edaa7c2211f1b6dc00155d001a19</value>
  </attribute>
 ▼<attribute isleave="true" leave_typeid="string" name="RequiresSession">
    <value index="0" isnull="false">true</value>
  </attribute>
 ▼<attribute isleave="true" leave_typeid="string" name="ServerApplicationId">
    <value index="0" isnull="false">localhost</value>
  </attribute>
 ▼<attribute isleave="true" leave_typeid="string" name="ServerLogicalSystemName">
    <value index="0" isnull="true"/>
  </attribute>
 ▼<attribute isleave="true" leave_typeid="string" name="ServiceId">
    <value index="0" isnull="false">checkmmeswcv</value>
  </attribute>
</instance>
```

Запрос к /rep/checkmmeswcv/int, заполнены:
ClientId, ClientLanguage, ClientLevel, ClientUser.
RequestId вероятно должен меняться от запроса к запросу.

MethodId = default и ServiceId = checkmmeswcv (из предыдущего слайда, ответ от getregisteredhmimethods).

MethodInput для данного сервиса+метода пуст.

Быстрый взгляд на instance и HmiResponse

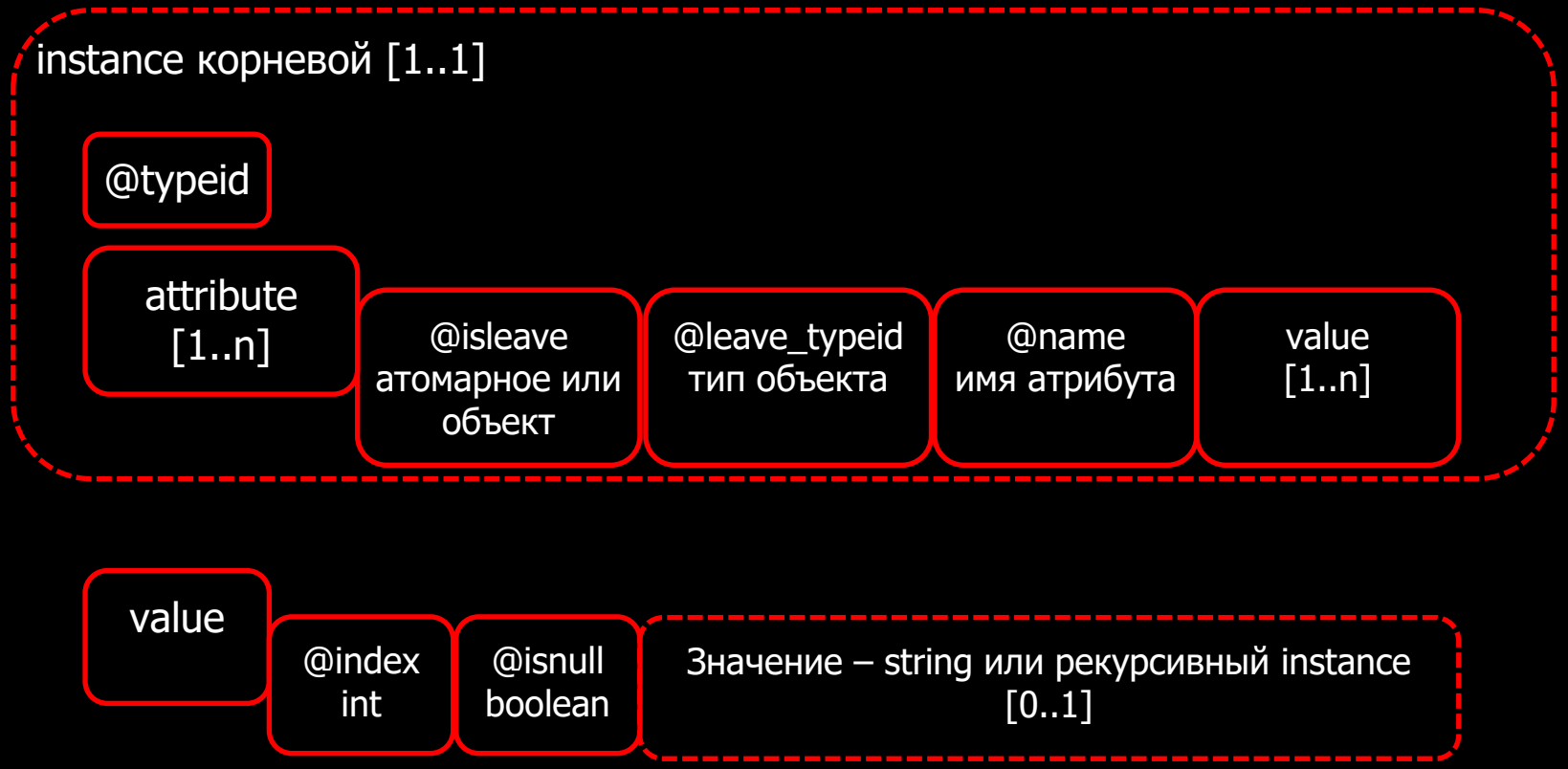


```
<!-- suppress ALL  -->
<instance typeid="com.sap.aii.utilxi.hmi.core.msg.HmiResponse">
 <attribute isleave="true" leave_typeid="string" name="ClientId">
   <value index="0" isnull="false">74f9edaa7c2211f1b6dc00155d001a19</value>
 </attribute>
 <attribute isleave="true" leave_typeid="string" name="ControlFlag">
   <value index="0" isnull="false">0</value>
 </attribute>
 <!-- suppress CheckTagEmptyBody  -->
 <attribute isleave="false" name="CoreException">
   <value index="0" isnull="true"/>
 </attribute>
 <attribute isleave="true" leave_typeid="string" name="HmiSpecVersion">
   <value index="0" isnull="false">1.0</value>
 </attribute>
 <!-- suppress CheckTagEmptyBody  -->
 <attribute isleave="false" name="MethodFault">
   <value index="0" isnull="true"/>
 </attribute>
 <attribute isleave="false" name="MethodOutput">
  <value index="0" isnull="false">
   <instance typeid="com.sap.aii.utilxi.hmi.api.HmiMethodOutput">
    <attribute isleave="true" leave_typeid="string" name="ContentType">
      <value index="0" isnull="false">text/xml</value>
    </attribute>
    <attribute isleave="true" leave_typeid="string" name="Return">
      <value index="0" isnull="false"><?xml version="1.0" encoding="UTF-8"?> <swcv_response><sw
    </attribute>
   </instance>
  </value>
 </attribute>
 <attribute isleave="true" leave_typeid="string" name="RequestId">
   <value index="0" isnull="false">74f9eda97c2211f1a63500155d001a19</value>
 </attribute>
</instance>
```

Смысловой ответ от /rep/checkmmeswcv/int

MethodOutput = строка
<swcv_response>
<swcv unique_id='' swcv_name='' swcv_state='Not Configured' />
</swcv_response>

HMI Instance.
Чтобы понять рекурсию, надо понять рекурсию.



Низкоуровневый формат, в который сериализуется любая HMI структура (и наоборот).

Схема: hmi.xsd

HMI-имба



- SimpleQuery это сервис /rep/query/int или /dir/query/int но с UTF-8
- Если мы прикинемся HMI-сервером для абапа, то получим SAP_PROXY_ESR SPROXY (правда, там много возни с реализацией навигации в дереве /rep/goa/int и XSD / WSDL)
- /rep/read/int это чтение кусков xiObj (xsd, wsdl, xim …) по набору критериев
- /rep/start/repository.jnlp это тоже HMI
- /rep/transportservice/int может показать переносы за период
- /rep/mappingtestservice/int - можно запускать тесты OperationMapping / MessageMapping снаружи, без объектов Directory
- /rep/write/int вероятно может создавать объекты (я не проверял)
- Крайне удобно снимать пейлоады, ставя HTTP-прокси между NWDS и SAP PO

Общие мысли, риторические вопросы

Выглядит так, что TPZ пропахан, xiObj засеян а на поле HMI бурьян.
По фронтенду и генерации HTML конь не валялся.

К чему всё это на финише SAP PO? Будущее покажет!

Спасибо за внимание!

KartaDSL – попытка сгенерировать на Java описание всего



```
from("seda:ordersInput")
.filter().simple("${body.amount} > 100")
.to("seda:outputQueueA");

from("timer://dbPoller?period=5000&fixedRate=true")
.log("Запуск опроса базы данных")
    // Читаем из базы данных
    .setBody(constant("SELECT * FROM orders WHERE processed = false"))
    .to("jdbc:myDataSource?outputType=SelectList&outputClass=java.util.Map")
    .log("Получено ${body.size()} записей из БД")
    .process(convertToXml)
    .choice()
     // Ветка 1: Если элементов > 100, шлем в hundred
     .when().xpath("count(/orders/order) > 100")
         .log("Найдено ${xpath:count(/orders/order)} элементов (больше 100)")
         .setHeader("CamelHttpMethod", constant("POST"))
         .setHeader("Content-Type", constant("application/xml"))
         .to("http://hundred/api/orders")
         .log("Отправлено в http://hundred")
     // Ветка 2: Все остальные шлем в constant
     .otherwise()
         .log("Найдено ${xpath:count(/orders/order)} элементов (не более 100)")
         .setHeader("CamelHttpMethod", constant("POST"))
         .setHeader("Content-Type", constant("application/xml"))
         .to("http://constant/api/orders")
         .log("Отправлено в http://constant")
    .end();
```

```
from("sapjdbc:P_PARTY|BS_DMS|CC_JDBCSender_Tags")
.setAddress("{urn:dummy_namespace}SI_Tag_OutAsync")    пишем интерфейс икохи
.filter().xpath("count(//tag) > 100")
.transform("{urn:mappings}OM_Mapping")
.setAddress("{urn:smart_namespace}SI_Tag_InAsync")
.to("sapxi:ABD100|CC_XIRecv");

//Это описание конфига канала:
config("sapxi:ABD100|CC_XIRecv")
.set("HTTPdestination", "XI_SOAP_RECEIVER_ABD");

//Это описание конфига NWA HTTP Dest:
config("sapnwa:destination:XI_SOAP_RECEIVER_ABD")
.set("targetUrl", "http://abd100/sap/xi/engine")
.set("auth", "BASIC")
.set("login", "techuser")
.setPasswordUsingKeypass("в кипасе лежит как XI_SOAP_RECEIVER_ABD_USER");
```

Слева CamelDSL, справа предполагаемый KartaDSL, оба являются Java-кодом

Спасибо!



Контакты:

Илья Кузнецов
Консультант по интеграции, Август
iliya.kuznetsov@gmail.com
+7929-9533259